

# 大数据技术与基础

## 课后作业

学号：2106410112

姓名：吴天宇

班级：计算机2101

抓取一个公开网站上的数据，数据必须是多页，存为 **csv** 格式；如果数据是多个 **pdf**，将其合并为一个 **pdf**。程序要尽量自动化，减少人为干预。

代码：

```
import requests
import json
import csv
all_data=[]
def Get_data():
    header = {
        'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124
Safari/537.36'
    }
    url = 'https://api.bilibili.com/x/web-interface/popular'
    for i in range(1, 25):
        param = {
            'ps': 20,
            'pn': i
        }
        res = requests.get(url=url, headers=header, params=param)
        # print(res.text)
        Parse(res.text)
        res.close()
def Parse(text):
    dict=json.loads(text)
    # print(dict['data']['list'])
    for i in dict['data']['list']:
        # print(i['title']) # 标题

        # print(i['owner']['name']) # up 主

        # print(i['stat']['view']) # 播放量

        # print(i['stat']['danmaku']) # 评论数

        temp = (i['title'],i['owner']['name'],i['stat']['view'],i['stat']
['danmaku'])
        all_data.append(temp)
```

```

def Save_data(all_data):
    with open(r'data.csv',mode='w',encoding='utf-8') as fp:
        writer = csv.writer(fp)

        writer.writerow(['视频名称', 'up 主', '播放量', '评论数'])

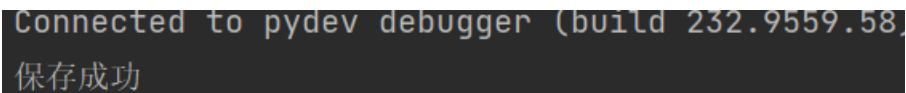
        for data in all_data:
            writer.writerow(data)
if __name__ == '__main__':
    Get_data()
    # print(all_data)
    Save_data(all_data)

```

程序说明：

此脚本使用请求库向指定的 URL 发出 GET 请求。然后，它从响应中提取数据，并将其保存为 csv 文件。提取的数据为 B 站热门视频的视频名称, up 主, 播放量, 评论数。

运行截图：



```

Connected to pydev debugger (build 232.9559.58)
保存成功

```

视频名称,up主,播放量,评论数

答非所问（奥义超级版）,小潮院长,1003261,18872

《原神》角色演示-「莱欧斯利：即兴发挥」,原神,1600748,7669

跟着福岛渔民出海，捞上来的鱼检测结果如何？,食贫道,8452831,112330

不多说了，请看vcr,李炮炮儿,1677519,2230

“你有钱”.....别酸！孩子涨见识和花钱，没多大关系！,侃木头哦,440895,269

领导说干不了就滚,逗比的雀巢,4871300,3342

逼疯各地域老板第三弹！爱吃胡辣汤的有福了！,陈嘉伦整个路人,1347447,1443

遗憾终究是遗憾,大圆哥\_,2404013,7034

钢铁猛片！他妈让他出人头地，男孩听了进去，一代枭雄的崛起！,火龙果讲电影,1544625,4476

【独家】傲世九重天 第四话 弟子之争【10月国创】,哔哩哔哩国创,932355,751

**葡萄酒数据集（wine.data）**搜集了法国不同产区葡萄酒的化学指标。试建立决策树、随机森林和 **xgBoost3** 种分类器模型，比较各种分类器在此数据集上的效果。

**【提示】：**每种分类器，需要对参数进行尝试，找出此种分类算法的较优模型，再与其他分类器性能进行比较。

代码：

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
```

```
# 读取数据集
```

```
data = pd.read_csv('wine.data', header=None)
```

```

# 将数据集分为特征和标签
X = data.iloc[:, 1:]
y = data.iloc[:, 0]
from sklearn.preprocessing import LabelEncoder

#XGBoost 要求目标变量必须是从 0 开始的整数，而 y 变量取值是 [1, 2, 3],

使用 LabelEncoder 进行转换

le = LabelEncoder()
xgy = le.fit_transform(y)

# 将数据集分为训练集和测试集

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
X_train, X_test, xgy_train, xgy_test = train_test_split(X, xgy,
test_size=0.3, random_state=42)

# 建立决策树模型

dtc = DecisionTreeClassifier(random_state=42)
dtc.fit(X_train, y_train)
dtc_score = dtc.score(X_test, y_test)

# 建立随机森林模型

rfc = RandomForestClassifier(random_state=42)
rfc.fit(X_train, y_train)
rfc_score = rfc.score(X_test, y_test)

# 建立 xgBoost 模型

xgb = XGBClassifier(random_state=42)
xgb.fit(X_train, xgy_train)
xgb_score = xgb.score(X_test, xgy_test)

# 输出各个模型在测试集上的表现

print('Decision Tree Classifier score:', dtc_score)
print('Random Forest Classifier score:', rfc_score)
print('XGBoost Classifier score:', xgb_score)

```

程序说明：

从 wine.data 中读取数据将数据集中除第一列外的所有列赋值给变

量 X，将数据集中第一列赋值给变量 y。使用 LabelEncoder 进行对 y 的值进行转换，将数据集分为训练集和测试集，test\_size 为 0.3,random\_state 为 42,分别建立三种模型，并对模型的性能进行测试，输出各个模型在测试集上的表现，发现随机森林的模型最好

运行截图

```
Decision Tree Classifier score: 0.9629629629629629  
Random Forest Classifier score: 1.0  
XGBoost Classifier score: 0.9629629629629629
```